

# **c R A R k for 7-Zip**

## **(Fastest 7-Zip Cracker)**

### **v. 2.0**

(c) Copyright PSW-soft 2012-20 by P. Semjanov

THIS PROGRAM VERSION IS DISTRIBUTED "AS IS". YOU MAY USE IT AT YOUR OWN RISK. ALL THE CLAIMS TO PROGRAM OPERATION WILL BE REJECTED. THE AUTHOR DOES NOT ALSO GUARANTEE THIS PROGRAM FUTURE MAINTENANCE AND UPDATE.

This is FREeware program, so it can be distributed under the following conditions: program code is kept unchanged and the program is distributed in the form of distributive archive. Any commercial use of this program is prohibited!

1. PURPOSES AND CHARACTERISTICS
2. REQUIREMENTS FOR THE INPUT ARCHIVE
3. Working with the program
4. THE USE OF PASSWORD DEFINITION FILE
  - 4.1. Password definition file format
  - 4.2. Password definition
    - 4.2.1. Character sets
    - 4.2.2. Dictionary words and their modifiers
    - 4.2.3. Permutation brackets
  - 4.3. Dictionaries and character sets definitions
    - 4.3.1. Dictionaries definition
    - 4.3.2. Definition of the character sets in use
    - 4.3.3. Definition of conversion modifiers
    - 4.3.4. Special character sets definition
  - 4.4. USEFUL EXAMPLES OF PASSWORD DEFINITION
5. Possible problems (FAQ)
6. On PDL library
7. How to contact the author
8. Special thanks

## **1. PURPOSES AND CHARACTERISTICS**

The cRARk for 7-Zip program is designed to determine a forgotten password for 7-Zip archives. It supports 7-Zip archives versions from 3.x and up.

To proceed with cRARk-7z program you need processor with SSE2 support. It is recommended to use as powerful processor as possible (the code is optimized for Core 2/Core iX/AMD Bulldozer). From v. 1.0, cRARk-7z can use both NVIDIA and AMD/ATI GPUs with CUDA/OpenCL support.

cRARk-7z is the tool for professionals, no GUI or great service is provided. But it tries to maximize your abilities for passwords definition and to minimize search time. cRARk-7z uses Password Cracking Library (PDL), a very powerful tool allowing you to define rules to generate passwords.

The rate of password search should be about of 150 password per second on modern powerful computer, so finding the 6-characters password of lower case Latin letters will need about a month. In the GPU the rate could achieve 5000 p/s. Rate for dictionary attack is about 50 pass/sec (GPU is not supported ).

## 2. REQUIREMENTS FOR THE INPUT ARCHIVE

Program should work with 7-Zip archive if following requirements are to be met:

- There is at least one encrypted file.
- The password should be not longer than 25 symbols (on GPU engine).

In case of solid-archives, the first file should satisfy these requirements. Therefore, if the files in archive were encrypted with the different passwords, the password for first file will be found.

cRARk-7z must be working with selfextracting (sfx) and multivolume archives, as well as archives with encrypted headers.

From version 1.0, cRARk-7z supports passwords computations on GPU cards (OpenCL technology). Current limitations are:

- Only NVIDIA cards with computing capability 3.0 and up is supported. AMD/ATI GPUs are supported from 5xxx/6xxx serie.
- You need the latest drivers with CUDA and/or OpenCL support
- CUDA and OpenCL engine is activated if password definition contains '\*' (repetition symbol). All another passwords are tested on CPU.

## 3. Working with the program

To run the program YOU ARE TO CREATE PASSWORD DEFINITION FILE firstly (see [section 4](#)).

This is a command-line utility! You should run it from command (DOS) prompt.

To run the program you should use:

```
cRARk-7z [options] archive
```

The found password is printed in such a form:

```
truepass - CRC OK
```

Next it is repeated in hexadecimal PDL-like form (see [section 4.2.1](#)).

All other messages ARE NOT passwords and are intended as progress indication of the program.

Options in this mode are:

|        |                                                                                                                                                                                                                                                |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -lXX   | set password length to XX at least (XX = 0..255, XX = 1 by default). This parameter affects password length only when '*' is used in its definition (see <a href="#">section 4.2.1</a> );                                                      |
| -gXX   | set password length to XX at most (XX = 0..255, XX = 8 by default)                                                                                                                                                                             |
| -pXXXX | set the name of password definition file ("PASSWORD.DEF" by default).                                                                                                                                                                          |
| -b     | perform benchmarking                                                                                                                                                                                                                           |
| -v     | debug mode (see <a href="#">section 5</a> ). It may be used to show character sets in use. This option generates also all the passwords according to with their definition; it does not test but prints them, so you can check their validity. |
| -c     | disable GPU support (use CPU only)                                                                                                                                                                                                             |
| -nX    | use GPU #X                                                                                                                                                                                                                                     |
| -dXX   | set GPU delay (XX = 0..5), 0 - fastest, 5 - slower, but smoothest                                                                                                                                                                              |
| -fWXY  | (advanced) set crypto functions #W,X,Y. Some information about available functions is printed by -b option.                                                                                                                                    |

## 4. USING OF PASSWORD DEFINITION FILE

Password definition file is the main control file. Its translation and processing are the main task of PDL library. Its format doesn't depend on application, to which PDL is linked, so this library can be used for any password searching program.

Password definition language (PDL) is designed for flexible and powerful control of password generation. It allows password definition creation using all known information about a password which substantially reduce the time of password searching. The typical example of such information is : "I remember that my password consist of two words separated by the one of the signs "-", "\_", "=", ",", and the first letters of the words are in the uppercase" (see [example 1](#) to see how it looks in PDL).

The main idea of PDL language is component-based description of password. The above example has three components - first word, separator (symbol) and the second word. In most cases is it possible to divide the password verbal description into components.

The syntax of PDL language is close to the regular expressions syntax. Your knowledge of regular expressions will help you understand PDL language easily.

There are three main components type in PDL language - [charsets](#), [words](#) and [generators](#).

### 4.1. Password definition file format

Password definition file is just text file in UTF-16 or UTF-8 format (Windows) or UTF-8 (Linux, Mac OS) and consists of two parts: firstly, [dictionary and character set definition](#), and secondly, [passwords definition](#).

The parts are separated by a line of two '##' symbols:

```
[ <dictionary and character set definition> ]
##
```

<passwords definition>

The first part could be missing, in that case password definition file starts with '##' symbols.

In all other cases the symbol '#' is considered as a start of comment.

Space characters and tabs are ignored in password definition file and may separate any components.

To make it easy, let's look upon the password definition mechanism at first and then character set definition, contrary to their position in password definition file.

## 4.2. Password definition

This is the main part of the file. IT NECESSARILY EXISTS IN ANY PASSWORD DEFINITION FILE (PASSWORD.DEF) AFTER THE LINE '##' and presets password generation rules to be checked out later on. It consists of the text lines, each of them gives its own password set and mode of operation, i.e. an algorithm of password search. Each line is independent and is processed separately, herewith the total number of checked passwords is counted up.

Character sets, dictionary words and generators form password definition. They preset one or more characters, which will hold the appropriate position in a password.

### 4.2.1. Character sets

Character set (charset) is a range of characters, any of them can occupy current position in a password. The supported charsets are:

1) Simple single characters (**a**, **b**, etc.). It means this particular character occupies given position in a password;

2) Shielded characters. If any of special characters can ever occur in the password, it must be shielded with '\'. The meaning is identical with item 1 mentioned above. Among these characters are:

|                        |                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------|
| \\$, \., \*, \?, \=    | '\$', '., '*', '?', '=                                                                              |
| \], \[, \{, \}, \[, \) | corresponding brackets;                                                                             |
| \ (space character)    | space character                                                                                     |
| \xxxx                  | any Unicode character in hex (X is a hexadecimal digit), like \D9E9                                 |
| \0                     | no character. It is usually used in conjunction with "real" character (please find examples below). |

Generally, any character can be shielded except hexadecimal digits.

3) Macros of character set. It means that current position in the password can be occupied by any character from the set. These sets are specified in the first part of password definition file (see [section 4.3.2](#)) and are denoted as:

|            |                                                                                   |
|------------|-----------------------------------------------------------------------------------|
| <b>\$a</b> | lower-case Latin letters (26 letters, unless otherwise specified)                 |
| <b>\$A</b> | upper-case Latin letters (26 letters, unless otherwise specified)                 |
| <b>\$!</b> | special characters (32 characters, unless otherwise specified)                    |
| <b>\$1</b> | digits (10 digits, unless otherwise specified)                                    |
| <b>\$i</b> | lower-case letters of national alphabet                                           |
| <b>\$I</b> | upper-case letters of national alphabet                                           |
| <b>\$o</b> | other user-specified characters                                                   |
| <b>?</b>   | any character (i.e. all the characters, included into the macros mentioned above) |

NOTE: macros **\$v** and **\$p** (see [section 4.3.4](#)) cannot be used for password definition.

4) Any combinations of the characters mentioned above. It must be written in square brackets. The meaning is identical with item 3 mentioned above. For example:

|                                      |                         |
|--------------------------------------|-------------------------|
| <b>[\$a \$A]</b>                     | any Latin letter        |
| <b>[abc]</b>                         | a, or b, or c           |
| <b>[\$1 abcdef]</b>                  | hexadecimal digit       |
| <b>[s \0]</b>                        | s or nothing            |
| <b>[\$a \$A \$1 \$! \$i \$I \$o]</b> | this is equivalent to ? |

5) Regular repetition character **\***. It means the previous character set has to be repeated in related item of the password 0 or more times. For example:

|                                  |                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------|
| <b>\$a *</b>                     | a password of any length, consisting of lower-case Latin letters                    |
| <b>[ab] * c</b>                  | c, ac, bc, aac, abc, bac, bbc, aaac, ...                                            |
| <b>[\$a \$A] [\$a \$A \$1] *</b> | "identifier", i.e. a sequence of letters and digits with a letter at first position |

Note that password of zero length (null password) is physically meaningful and is not always the same as no password at all.

The length of repetition is computed in two ways:

- automatically on the base of the defined maximum and minimum password length options. (Note, when there is no '\*' in password definition, these parameters are ignored.)
- set explicitly in brackets in such way: **\* (length)** or **\* (min length, max length)**. Examples:

|                      |                                                 |
|----------------------|-------------------------------------------------|
| <b>a b *(0,3) c</b>  | will generate 4 passwords: ac, abc, abbc, abbbc |
| <b>a [bc] *(2) d</b> | abbd, abcd, acbd, accd                          |
| <b>\$1 *(2,4)</b>    | all two-, three- and four-digit numbers         |

It is recommended to use '\*' as wide as possible. This is because it allows to perform the most powerful search. Although the constructions '?\*' and '? ?\*' seem to be alike from the logic standpoint, the first one will be searched through faster.

Current limitation: '\*' can be used only once.

#### 4.2.2. Dictionary words and their modifiers

Contrary to the character set, the words present several consecutive passwords characters. Two dictionaries are supported by PDL library: main (with ordinary words mainly) and user's (where special information can be stored, for example, proper names, dates, etc.), though there is no difference between them.

Dictionary is a text file in both UTF-16 or UTF-8 encoding (Windows) or only UTF-8 (Linux, Mac), consisting of the words, separated by the end-of-line characters. Both DOS-format (CR/LF) and UNIX-format (LF) files can be used. Preferably to use words of the same (lower) case in dictionaries (to increase search rate, among other factors). To sort out words by its length is also desirable.

Thus, there are two macros:

|            |                                 |
|------------|---------------------------------|
| <b>\$w</b> | a word from the main dictionary |
| <b>\$u</b> | a word from the user dictionary |

As is known altered words are often used as passwords. So a whole set of word modifiers is introduced to determine such passwords. Among these are:

|                       |                                    |
|-----------------------|------------------------------------|
| <b>.u</b> (upper)     | to upper-case                      |
| <b>.l</b> (lower)     | to lower-case                      |
| <b>.t</b> (truncate)  | to truncate up to the given length |
| <b>.c</b> (convert)   | to convert the word                |
| <b>.j</b> (joke)      | to upper-case some letters         |
| <b>.r</b> (reverse)   | to reverse the word                |
| <b>.s</b> (shrink)    | to shrink the word                 |
| <b>.p</b> (duplicate) | to duplicate the word              |

Modifiers may have parameters, written in round brackets. For modifiers, meant for single letters use, it is possible to preset a number of the letter as a parameter. Lack of parameters or null parameter means "the whole word". Then, letters can be numerated from both the beginning of the word and the end of the word. The end of the word is denoted with the character '-' .

There are only three such modifiers for today: **.u**, **.l**, **.t**. So, use:

|                                 |                                                                          |
|---------------------------------|--------------------------------------------------------------------------|
| <b>.u</b> or <b>.u(0)</b>       | to upper-case the whole word (PASSWORD)                                  |
| <b>.u(1)</b> ,<br><b>.u(2)</b>  | to upper-case only the first (the second) letter<br>(Password, pAssword) |
| <b>.u(-)</b> ,<br><b>.u(-1)</b> | to upper-case the last (the next to last) letter<br>(passworD, passwoRd) |
| <b>.t(-1)</b>                   | to truncate the last letter in the word (passwor)                        |

The other modifiers operate with the whole words only and their parameters give the way of modification. The following modifier parameters are specified for today:

|                           |                                       |
|---------------------------|---------------------------------------|
| <b>.j(0)</b> or <b>.j</b> | to upper-case odd letters (PaSsWoRd)  |
| <b>.j(1)</b>              | to upper-case even letters (pAsSwOrD) |

|                           |                                                                                                                            |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <b>.j(2)</b>              | to upper-case vowels (pAsswOrd)                                                                                            |
| <b>.j(3)</b>              | to upper-case consonants (PaSSWoRD)                                                                                        |
| <b>.r(0)</b> or <b>.r</b> | to reverse the word (drowssap)                                                                                             |
| <b>.s(0)</b> or <b>.s</b> | to reduce the word by discarding vowels unless the first one is a vowel<br>(password -> psswrd, offset -> offst)           |
| <b>.p(0)</b> or <b>.d</b> | to duplicate the word (passwordpassword)                                                                                   |
| <b>.p(1)</b>              | to add reversed word (passworddrowssap)                                                                                    |
| <b>.c(&lt;number&gt;)</b> | to convert all the letters in the word according to the appropriate conversion string (see <a href="#">section 4.3.3</a> ) |

All the modifiers operate adequately with both Latin and national letters, provided that the rules of national character sets definition are observed. Clearly there can be more than one modifier (the number of consecutive modifiers is limited by 63, which is unlikely to be exceeded). For example: (let \$w mean a "password"):

|                      |          |
|----------------------|----------|
| <b>\$w.u(1).u(-)</b> | PassworD |
| <b>\$w.s.t(4)</b>    | pssw     |
| <b>\$w.t(4).s</b>    | pss      |

### 4.2.3. Generators

Generator is the last possible component type and it generates several passwords of different length from the given symbols in current position. Generators are denoted by { and } symbols followed by generator type, like {abc}.u. The opening bracket indicates the position of beginning the generator, and the closing one - the ending position.

- 1) The default generator - permutation brackets { }

This generator has no explicit type, i.e. it simply denoted by { and }

The problem is widely met, when you remember your password, but it is not valid for some reason. Probably, you mistype it. The PDL engine has its own algorithm to restore such passwords. The following typing mistakes are considered: two neighboring letters are swapped (psasword), a letter is omitted (pasword), an needless letter is inserted (passweord) or one letter is replaced with another (passwird). Such password changes will be referred to as permutations.

To indicate the beginning and the end of the password section where permutations could appear, permutation brackets '{' and '}' are used. The bracket '}' can be followed by a number of permutations (1 by default), separated by a dot. The physical meaning of the number of permutations is the number of simultaneously introduced mistakes. For example:

{abc} - 182 (different) passwords will be obtained, including:

|                       |                       |
|-----------------------|-----------------------|
| <b>bac, acb</b>       | 2 swaps               |
| <b>bc, ac, bc</b>     | 3 omissions           |
| <b>aabc, babc ...</b> | 4 * 26 - 3 insertions |
| <b>bbc, cbc ...</b>   | 3 * 25 replacements   |

|     |                 |
|-----|-----------------|
| abc | the word itself |
|-----|-----------------|

**{password}.2** - the following words will be generated: psswrod, passwdro, paasswor, etc.;

**{\$w}** - all the words, containing one mistake, from the main dictionary.

Notes:

a) It is obvious that some passwords will be obtained more than once, so the larger is the number of permutations, the larger is the number of replicas. Efforts were made in this program to reduce replicas, but they are purely empirical and were made for two permutations at most. In other words, for the large numbers there is no certainty that a particular password cannot be discarded erroneously.

b) For insertion and replacement one should know the set of characters to be inserted or replaced. In the event this set is not specified explicitly (see [section 4.3.4](#)), this program forms it automatically for character sets, in relation to standard set these characters are from (i.e. for **{password}** \$a will be inserted, for **{Password}** [\$a \$A] will be inserted). The similar operation with words is performed, based on the first word from the dictionary with modifiers being taken into account. In the event this set is specified explicitly, it is just the set to be used.

2) Upper and lowercase generators **{}.u**, **{}.l**

These generators will generate all possible upper or lower-case combinations from the given symbols, like:

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| <b>{abc}.u</b> | abc, Abc, aBc, abC, ABc, AbC, aBC, ABC                            |
| <b>{AbC}.l</b> | AbC, abC, Abc, abc                                                |
| <b>{\$w}.u</b> | All main dictionary words with all possible upper-case conversion |

3) Conversion generator **{}.c**

Like the above generator, this one is used to generate all possible combination of word using the appropriate convert table (see [section 4.3.3](#)) written in round brackets.

Let **.c(0)** convert table defines conversion of letters to similar symbols, then

|                        |                                                                                        |
|------------------------|----------------------------------------------------------------------------------------|
| <b>{password}.c(0)</b> | password, pa\$sword, pas\$word, passw0rd, pa\$\$word, pa\$sw0rd, pas\$w0rd, pa\$\$w0rd |
|------------------------|----------------------------------------------------------------------------------------|

Of course, there may be several generators in the password description. So, the definition like

**{my{good}password}.u**

may occur.

## 4.3. Dictionaries and character sets definitions

All the definitions are set in the beginning of password definition file up to the characters '###'.

### 4.3.1. Dictionaries definition

The main and user dictionaries in use (see [section 4.2.2](#)) are initially

defined as usual. It is necessary only if you are going to use words from the dictionaries when defining passwords, i.e. \$w or \$u.

The dictionaries are given as follows:

|                                         |                                |
|-----------------------------------------|--------------------------------|
| <code>\$w = "main.dic"</code>           | <code># main dictionary</code> |
| <code>\$u = "c:\\dict\\user.dic"</code> | <code># user dictionary</code> |

File name is to be quoted, the path characters are to be shielded.

#### 4.3.2. Definition of the character sets in use

The character sets in use are defined as usual. They are classified in two groups: predefined and user-defined.

Predefined sets include:

|                  |                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------|
| <code>\$a</code> | lower-cased Latin letters, 26 letters in all                                                                |
| <code>\$A</code> | upper-cased Latin letters, 26 letters in all                                                                |
| <code>\$!</code> | special characters (32 characters in all)<br>{ } : " < > ? [ ] ; ' , . / ~ ! @ # \$ % ^ & * ( ) _ + ` - = \ |
| <code>\$1</code> | digits, 10 digits in all                                                                                    |

User-defined sets include:

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| <code>\$i</code> | lower-cased letters of national alphabet                           |
| <code>\$I</code> | upper-cased letters of national alphabet                           |
| <code>\$o</code> | additional character set (for example, any non-typable characters) |

Character sets are defined as follows:

**`$<charset> = [ <single characters or character sets> ]`**

In other words, character set is written as combination of characters (see [section 4.2.1](#)), for example:

`$i = [$a ä ö ü ß ]`  
`$o = [$! $1 \20ac]`

NOTES:

1) Any character sets are allowed to be defined, including pre-defined. For example, you may include additional characters, such as euro sign € (\20ac) into the set \$!

2) When the sets \$i and \$I are being defined, the function of switching between lower/upper case is defined automatically. So it is important to have letters being ordered uniformly in these sets.

The full character set '?', consisting of [\$a \$A \$1 \$!\$i \$I \$o] (just such an order is of importance in the next section), is never formed until all the characters are defined.

#### 4.3.3. Definition of conversion modifiers

The conversion modifiers `.c` may be defined (see [section 4.2.2](#)) are described the conversion rule of one character to another. It is performed with the line of the form: `.c(<number>) = "<conversion string>"`

The conversion string is written in such way `"o=0|l=1|s=5|b=6"`,

where the conversion pairs are divided with '|' symbol and at left side of '=' is written a character to be converted to character on the right side of '='.

For example, let the \$w is "lower", then \$w.c(0) = "l0wer". The characters '\' and '|' are to be shielded in conversion string. The numbers of modifiers may vary from 0 to 255.

#### 4.3.4. Special character sets definition

Among special character sets are:

|            |                                                                                                                                                                                            |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>\$v</b> | a set of vowels (in all alphabets being used). It is needed only when .s and .j modifiers are used                                                                                         |
| <b>\$p</b> | a set for insertion and replacement for permutation brackets. It is needed only if automatic generation of this set does not suit you for some reason (see <a href="#">section 4.2.3</a> ) |

These sets are defined in a similar way to the other character sets.

#### 4.3.5.Minimal and maximal password length

The minimal and maximal password length settings affect only password definition with [regular repetition symbol](#) and not affected other password definition including simply charsets, words or generators. To set the default password range, use keywords **min** and **max**, like:

|                |                                            |
|----------------|--------------------------------------------|
| <b>min = 1</b> | generate passwords from one character long |
| <b>max = 5</b> | generate passwords up to 5 characters long |

### 4.4. USEFUL EXAMPLES OF PASSWORD DEFINITION

1) I remember that my password consist of two words separated by the one of the signs "-", "\_", "=", ",", and the first letter of the words are in the uppercase

```
$w = "dictionary.txt"
##
$w.u(1) [\-_|=,] $w.u(1)    # "-", "=" needs to be shielded
```

It should be mentioned that both \$w are distinct words, so a total of 20000 \* 4 \* 20000 = 1 600 000 000 passwords (if there are 20000 dictionary words) will be generated.

If you're not sure if first letter be in uppercase or not, you need to make all possible combination writing four-line definition:

```
$w = "dictionary.txt"
##
$w.u(1) [\-_|=,] $w.u(1)
$w.u(1) [\-_|=,] $w
$w [\-_|=,] $w.u(1)
$w [\-_|=,] $w
```

Additionally, if you may insert from 1 to 3 symbols between the words, the definition is changed as:

```
$w = "dictionary.txt"
##
$w.u(1) [\-_|=,] *(1,3) $w.u(1)
```

2) Most common definition - brute-force search using only lower-case Latin letters.

```
$a *
```

You also can use **min=** and **max=** settings or **-l** and **-g** options to define password length.

3) The same as the above, but using both cases Latin letter

```
[$a $A] *
```

4) Simplest dictionary attack

```
$w = "dictionary.txt"  
##  
$w
```

5) Dictionary attack with all possible upper/lower-case combinations:

```
$w = "dictionary.txt"  
##  
{ $w }.u
```

5) Let me cite ZEXPL2L program specification: "Let you have an archive with the password looking like "Heaven!!!", but you have forgotten, how many !s were there in the end and what kind of letters lower- or upper-cased were used: "HeAvEn!!!", "Heaven!" or "HeAven!!!!". But fortunately you remember your password to be 10 characters at most and 7 characters at least." This password will be written in PDL language as follows:

```
min = 7  
max = 10  
##  
{heaven}.u ! *
```

Instead of **min=** and **max=** **-l** and **-g** option can be used.

Suppose that among other things you have mistaken while typing the main part of the password. So the following one is worth attention:

```
min = 7  
max = 10  
##  
{{heaven}}.u ! *
```

6) One more citation from the same specification: "Let you have two variants of the password string: "myprog", "MyProg", "my\_prog" and "My\_Prog". It will be written as:

```
[mM] y [_ \0] [pP] rog
```

7) Password consists of exactly six letters from national alphabet:

```
$i $i $i $i $i $i
```

But

```
$i *(6)  
are far more efficient.
```

8) The date in DDMMYYYY format:

```
$1 * (8)
```

But

```
##  
[0123] $1 [01] $1 [12][90] $1 *(2)  
will generate much less passwords.
```

9) You remember your password to be "MyVeryLongGoodPassword", but it

is not valid for some reason. Try to use the following ones:

|                                   |                         |
|-----------------------------------|-------------------------|
| <b>{MyVeryLongGoodPassword}</b>   | 2360 passwords in total |
| <b>{MyVeryLongGoodPassword}.2</b> | 2 785 406 passwords     |

10) You know your password to be a meaningful word with a digit inserted elsewhere. The definition file is:

```
$p = [$1] # the insertion set is defined as a set of digits
##
{$w}
```

11) Syllable attack. You need to set up a dictionary of possible syllables of your language and then to search through all the meaningful words by proceeding as follows:

```
$u # monosyllabic words
$u$u # disyllabic words
$u$u$u # etc.
$u$u$u$u
```

12) In order to run your program in parallel on two computers, give them the following definition files:

|                              |                    |
|------------------------------|--------------------|
| <b>[abcdefghijklm] \$a *</b> | for the first one  |
| <b>[nopqrstuvwxyz] \$a *</b> | for the second one |

Proceed similarly with n computers.

## 5. Possible problems (FAQ)

1. How to break and then to continue the search.

The program may be broken painlessly once the message "Testing XX-chars passwords..." is displayed, and then the search may be continued with - lXX option (both XX are equal).

2. How to resume search from the password XXX?

Sorry, no way. It's implemented in the PDL 3.0 used in [Parallel Password Recovery](#).

3. The program has been searching for 10 days, but my password is not yet at hand.

Alas! It can't be helped. May be your password is too long, or the search set is wrong. Additional information on the password is necessary.

4. There are files with different passwords in the archive. What am I to do?

Just remove (using 7-Zip) files with already known passwords.

5. The software is not working with multivolume archives.

You need to concatenate all the parts of multivolume archive into one file, like: copy /b test.7z.001+test.7z.002+test.7z.033 test.7z

6. I have tested your program. To my mind, your program is nothing but utter error, it cannot even find "aaa2"-like password.

RTFM. Distributive file password.def searches through only lower-cased Latin letters. Change your password definition to "[ \$a \$1]

\*" and everything will be ok.

7. I've got beginning of one file from archive in plain text. Will it be useful to me?

No. At least, I couldn't use it. Could you? 7-Zip encryption sources are available in the official site.

8. I'd like to optimize your program. How can I get the sources?

You don't need them. Take 7-Zip sources and optimize the function in 7zAes.cpp file.

9. Is there any option to save program operation log?

Probably, you have never dealt with UNIX. Use:

cRARk-7z [options] > file

If you don't like this, use "tee" utility.

10. Is it possible to speed-up dictionary attack?

Yes, just sort your dictionary by the words size

11. Your distribution kit is packed with a password in itself!!! I do not find it funny!

You are reading this file, so you have solved this problem.

12. I need GUI, multicore support, pause/resume etc.

cRARk-7z is the free program, and I have no time to support such features. To find all you need, please look at [Parallel Password Recovery](#) which licensed cRARk-7z and PDL engine.

13. How to use my GPU card to recover passwords?

You need:

- a) NVIDIA GPU from GeForce 8 serie (some old 8800 cards may be not supported) or AMD Radeon HF from 5xxx
- b) latest drivers (it's recommended to get them from: [http://www.nvidia.com/object/cuda\\_get.html](http://www.nvidia.com/object/cuda_get.html)) and <http://support.amd.com/us/gpudownload/Pages/index.aspx>
- c) the password definition should contain '\*' symbol

14. I've got some errors when using my GPU card.

- a) Please install the latest drivers!
- b) Don't overclock neither GPU nor CPU!
- c) On Windows Vista and 7, if the message about the stopped driver appears, please use the file *driver-timeout.reg* from the distributon archive.

## 6. On PDL library

PDL 2.0 library is distributed by the author as FREeware in the form of source text. The reference to PDL as an obligatory requirement for your programs.

PDL 3.0 with plenty of new features is not a FREeware.

## 7. How to contact the author

Only by e-mail.

Please don't ask me about how to run the program etc - I have no time to explain individually.

e-mail: [pavel@semjanov.com](mailto:pavel@semjanov.com)  
WWW: <http://www.semjanov.com>

Program URL is: <http://crark.net/crark-7zip.html>

A lot of free, benchmarked password crackers you'll find at:

<http://www.password-crackers.com>

cRARK-7z is a FREE program, so all the claims will be rejected. Anyway, I'll be very grateful for pointing out the serious errors, such as:

- the program hangs up while searching (the lack of displayed messages is not an evidence of hangup);
- the program cannot find such password in such archive, although the set of characters in search is specified correctly

I'll be also glad to any constructive suggestions on improvements of program operation.

## **8. Special thanks and copyright notices**

To Igor Pavlov for 7-Zip, LZMA SDK and its source codes.

To Andy Polyakov and other OpenSSL guys for some optimization ideas.

This software uses cryptographic routines from CRYPTOGAMS: Copyright (c) 2006, CRYPTOGAMS by All rights reserved. Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met: \* Redistributions of source code must retain copyright notices, this list of conditions and the following disclaimer. \* Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution. \* Neither the name of the CRYPTOGAMS nor the names of its copyright holder and contributors may be used to endorse or promote products derived from this software without specific prior written permission. ALTERNATIVELY, provided that this notice is retained in full, this product may be distributed under the terms of the GNU General Public License (GPL), in which case the provisions of the GPL apply INSTEAD OF those given above. THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDER AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Good luck!

Pavel Semjanov, St.-Petersburg.